

Centrix — UML Diagrama de Atividades: Sistema de Consulta de Dados

Visão geral

Este documento descreve o **diagrama de atividades UML** do sistema Centrix, modelando o fluxo interno do sistema durante uma consulta de dados. Diferente do BPMN (que modela o processo de negócio), este diagrama modela o **comportamento do sistema**: como os componentes internos processam uma pergunta do usuário até entregar uma resposta formatada.

Swimlanes

O diagrama é dividido em três swimlanes, representando os componentes do sistema:

Swimlane	Responsabilidade
Interface	Interação com o usuário — recebe perguntas, exibe respostas e perguntas de clarificação
API	Orquestração do fluxo — validações (Pydantic), execução de SQL, injeção de schema, controle de retry
LLM	Inteligência — geração de SQL, formatação de resultados, elaboração de perguntas de clarificação

Fluxo principal (caminho feliz)

1. **[Interface]** Nó inicial → Usuário loga no APP
2. **[Interface]** Usuário envia pergunta/resposta
3. **[Interface]** Envia para API
4. **[API]** Valida a pergunta (Pydantic)
5. **[API]** Injeta o schema do BD no prompt
6. **[LLM]** Gera código SQL
7. **[API]** Merge node (ponto de convergência para retries)
8. **[API]** Roda código SQL
9. **[API]** Valida código SQL (validação de segurança — bloqueia comandos destrutivos como DELETE, DROP, UPDATE, etc.)
10. **[API]** Decisão: Código validado? → Sim
11. **[API]** Roda SQL no banco de dados
12. **[API]** Decisão: SQL rodou? → Sim
13. **[API]** Valida resultado (verifica se o resultado não é vazio, se os dados são consistentes, etc.)
14. **[API]** Decisão: Resultado validado? → Sim

15. [LLM] Formata resultado (transforma dados brutos em linguagem natural)
16. [API] Valida resultado formatado
17. [API] Decisão: Resultado formatado validado? → Sim
18. [Interface] Exibe resultado formatado ao usuário
19. [Interface] Nó final

Fluxo de retry (tentativas < 3)

Três pontos de falha convergem para um **merge node** de erro:

- Código SQL **não validado** (falha de segurança)
- SQL **não rodou** (erro de execução no banco)
- Resultado **não validado** (resultado vazio ou inconsistente)

Após o merge de erro:

1. [LLM] Decisão: Tentativas ≥ 3 ?
2. Se **Não** (tentativas < 3) → retorna ao merge node principal (passo 7) para nova tentativa, desta vez com o contexto do erro anterior para que a LLM repense a solução
3. Se **Sim** (tentativas ≥ 3) → segue para o fluxo de clarificação

Fluxo de clarificação (tentativas ≥ 3)

Quando a LLM esgota as 3 tentativas, ao invés de continuar chutando (o que gastaria tokens sem resultado), ela pede mais contexto ao usuário:

1. [LLM] Elabora pergunta ao usuário
2. [API] Valida pergunta
3. [API] Decisão: Pergunta validada?
 - **Sim** → [Interface] Envia pergunta ao usuário → Aguarda resposta → Usuário envia resposta (retorna ao passo 2 do fluxo principal)
 - **Não** → [API] Envia mensagem de erro à LLM → [LLM] Refaz pergunta → retorna à validação

Fluxo de retry da formatação

A formatação do resultado pela LLM também tem seu próprio ciclo de validação:

1. [LLM] Formata resultado (ou reformata, em caso de retry)
2. [API] Valida resultado formatado
3. [API] Decisão: Resultado formatado validado?
 - **Sim** → [Interface] Exibe resultado ao usuário

- **Não** → [API] Envia mensagem de erro → [LLM] Reformata resultado → retorna à validação

Elementos UML utilizados

Elemento	Notação	Uso no diagrama
Nó inicial	Círculo preenchido (●)	Início do fluxo — usuário loga
Nó final	Círculo com anel (⦿)	Fim do fluxo — resultado exibido / aguardando resposta
Action node	Retângulo arredondado	Todas as ações (validar, rodar SQL, formatar, etc.)
Decision node	Losango com condição	Decisões com caminhos Sim/Não
Merge node	Losango vazio	Convergência de caminhos (erros → retry, retries → pipeline)
Swimlane	Colunas verticais	Separação por componente (Interface, API, LLM)

Pontos de validação (resumo)

O sistema possui 5 pontos de validação distintos:

Validação	Componente	O que verifica	Consequência de falha
Valida pergunta (Pydantic)	API	Formato e estrutura da entrada do usuário	Rejeita input malformatado
Valida código SQL	API	Segurança — bloqueia comandos destrutivos (DELETE, DROP, etc.)	Conta como tentativa, retorna à LLM
Valida resultado	API	Resultado não vazio, dados consistentes	Conta como tentativa, retorna à LLM
Valida resultado formatado	API	Qualidade da formatação em linguagem natural	Retorna à LLM para reformatar
Valida pergunta (clarificação)	API	Qualidade da pergunta gerada pela LLM	Retorna à LLM para refazer

Decisões de design

- **Limite de 3 tentativas:** Evita consumo excessivo de tokens em loops infinitos. Após 3 falhas, a LLM pede mais contexto ao invés de continuar chutando.

- **Feedback de erro no retry:** Quando a LLM erra, ela recebe o erro como contexto adicional na próxima tentativa, permitindo que repense a solução.
- **Validação de segurança pré-execução:** O SQL é validado ANTES de rodar no banco, bloqueando qualquer comando que possa alterar dados.
- **Todas as validações na API:** A camada de validação é centralizada na API, mantendo a LLM focada apenas em geração de conteúdo.

Observações

- Este diagrama modela o **sistema** (UML), não o processo de negócio (BPMN). O BPMN correspondente está documentado em `BPMN_Consulta_Dados.md`.
- O diagrama cobre o fluxo completo incluindo autenticação, mesmo que o MVP não implemente login inicialmente.
- A modelagem dos dados (schema do banco) está documentada em `Schema_BD.md`.